



WantedBy

Business Specification

Build what people actually want.

Document type	Business Specification — v1.0
Product	WantedBy
Stage	Pre-build · Validation phase
Prepared by	Shahriyar Ahmed Mahir
Date	June 2026
Status	Draft — internal use

This document outlines the business objectives, target users, core pain points, product requirements, and success criteria for WantedBy — a real-time pain signal platform for indie hackers and side project builders.



01 — PRODUCT OVERVIEW

What is WantedBy?

WantedBy is a real-time product intelligence platform that continuously monitors Reddit, Hacker News, and developer communities for genuine product pain signals. It clusters semantically similar complaints, scores them by frequency and frustration intensity, and surfaces validated opportunity gaps to builders — so they stop guessing what to build and start building what people actually need.

The one-liner

Problem	Builders waste hours scrolling Reddit and HN trying to find what to build next.
Solution	WantedBy surfaces real, clustered, scored product pains from across the internet — automatically, in real time.
For	Indie hackers, side project builders, and early-stage founders who want demand-validated ideas before they write a line of code.
Tagline	Build what people actually want.

Why now

The number of people building side projects and indie products has exploded post-2022. Communities like r/SideProject (383k members), r/IndieHackers, and Hacker News see thousands of 'I wish someone would build X' posts every week. Nobody has built a clean, real-time tool that aggregates and structures this signal for builders. The closest competitors — manual Reddit scrolling, expensive market research tools, and gut instinct — are either time-consuming, costly, or unreliable.

02 — CORE USERS

Who we're building for

Primary user — The Weekend Builder

Who they are	Students, employed developers, and creatives who build side projects in their spare time. Not full-time indie hackers — people who have 5-10 hours a week and want to spend them building something people actually want.
--------------	---



Their situation	They have the skills to build. They lack a validated idea. They've shipped projects before that nobody used, and they don't want to repeat that mistake.
Age range	18 — 35
Background	Software engineering students, CS grads, junior-to-mid developers, bootcamp graduates
Time constraint	Limited — evenings and weekends only

Secondary user — The Early Founder

Who they are	People who want to start something but haven't committed to an idea yet. They're in 'exploration mode' — reading newsletters, watching founder videos, consuming startup content.
Their situation	They know they want to build a product but are paralysed by idea selection. They want validation before they invest months of effort.
Age range	22 — 40
Background	Ex-employees going indie, recent grads with entrepreneurial intent, non-technical founders partnering with developers

Tertiary user — The PM / Researcher

Who they are	Product managers and researchers at early-stage startups who need fast, cheap market signal without commissioning a formal research study.
Their situation	They need to understand what problems exist in a category. WantedBy gives them a live feed of real user frustrations organized by theme.
Age range	25 — 45
Background	Product managers, UX researchers, startup operators



03 — USER PAIN POINTS

What they struggle with today

Idea paralysis

The most common blocker. Builders have the skills but cannot decide what to build. They spend more time thinking about ideas than actually building.

Building for nobody

They ship a product after weeks or months of work and get zero users. The idea seemed good but was never validated against real demand.

Manual research is exhausting

The current process — scrolling Reddit, reading HN threads, searching Twitter — takes hours and yields inconsistent results. It's not repeatable or scalable.

No signal on gap size

Even when a builder spots a complaint online, they have no way to know if 100 people feel it or 100,000. There's no frequency or intensity data attached to individual posts.

Existing solutions are the wrong tool

Google Trends shows search volume, not frustration. Exploding Topics shows rising categories, not specific user needs. SurveyMonkey requires you to already know what to ask. None of them surface raw, unfiltered product demand from real conversations.

Fear of entering a crowded market

Builders often don't know what competitors exist for a given pain. They either avoid valid opportunities out of fear, or build into markets that are genuinely saturated without realising it.

04 — WHERE USERS LIVE

Where to find them

Online communities

Channel	Why it matters
r/SideProject	383k members. Primary community. 'I built this' posts, idea validation threads, feedback requests. Our highest-density target audience.



r/IndieHackers	Revenue-focused builders. More commercial intent. Receptive to tools that reduce wasted effort.
Hacker News	Technical founders and developers. 'Show HN' and 'Ask HN' threads. High-signal, high-trust audience.
r/Entrepreneur	Broader audience. Many people in early 'what should I build' phase.
r/webdev / r/programming	Developers who build side projects as a hobby. Highly technical, skeptical of hype.
Twitter / X	Indie hacker community is active here. Hashtag #buildinpublic has 100k+ regular participants.
Product Hunt	Launch platform. Our launch post here is part of the go-to-market plan.
Discord servers	Indie Worldwide, WIP.co, Maker Log — tight-knit communities of active builders.



05 — CORE REQUIREMENTS

What the product must do

Must-have (v1.0)

Continuous ingestion

Fetch posts and comments from Reddit (via PRAW) and Hacker News (via Algolia API) continuously, targeting frustration-signal phrases.

Pain signal classification

Classify each post as a genuine product pain signal or noise using LLM-based classification. Extract: category, core frustration summary, implied solution.

Semantic clustering

Group semantically similar pains together using vector embeddings (nomic-embed-text via Ollama) and Qdrant vector store, so 'invoicing is painful' and 'I hate sending invoices manually' are recognised as the same pain.

Gap scoring

Score each cluster by: frequency (how many mentions), recency (trending up or down), and competitive gap (are there existing solutions, and are they good?).

Discovery dashboard

A React + TypeScript dashboard showing the live pain feed, cluster explorer, and gap scores. Real-time updates via WebSocket.

User accounts

GitHub OAuth login. Free tier access to top 10 clusters per week.

Email capture

Landing page with email signup before full product is built. Validation-first.

Should-have (v1.1)

- Saved searches and custom alert emails when new clusters emerge in a category
- Trending view — pains rising fastest over the past 7 days vs previous 7 days
- Stripe billing — pro tier at \$12/month for unlimited clusters + API access
- Twitter/X ingestion as a third data source
- Competitor detection per cluster — auto-search Product Hunt and GitHub for existing solutions



- Weekly digest email with top 5 new clusters (Resend integration)

Out of scope for v1.0

- Mobile app
- Team collaboration features
- Slack / Discord bot integration
- Automated idea generation or AI-written product briefs
- CSV / Excel data exports (v1.2+)
- Kafka / Spark streaming infrastructure (architected for, not implemented in v1)



06 — INITIAL GOALS

What success looks like

Validation goals (before building)

Email signups	20+ signups from strangers (not friends/family) from the Reddit validation post
Post engagement	15+ upvotes on the r/SideProject validation post
DM responses	3+ positive replies from cold DMs to potential users
Target timeline	48 hours after the validation post goes live

Launch goals (week 8)

Registered users	50 users within 2 weeks of launch
Active users	20 users returning at least twice in the first month
GitHub stars	100+ stars on the open-source repo within 30 days
Reddit post performance	200+ upvotes on the launch post
Data volume	10,000+ pain signals ingested and classified
Uptime	99%+ during the first month post-launch

3-month goals

Paying users	10 pro subscribers at \$12/month (\$120 MRR)
Total users	200 registered accounts
Data volume	100,000+ posts processed
Community presence	Featured in at least one indie hacker newsletter

07 — GO-TO-MARKET

How we reach users



Phase 1 — Validation (this weekend)

- Post manual research findings on r/SideProject: '10 most repeated unmet product pains I found after reading 2 years of Reddit complaints'
- Include email signup link at the bottom of the post — do not pitch the tool directly
- Cold DM 10 people who posted about struggling to find ideas in the past month
- Post the same research insight on HN as a 'Show HN' or comment in relevant 'Ask HN' threads
- Deploy wantedby.vercel.app landing page with email capture live before posting

Phase 2 — Launch (week 8)

- Post 'I built a tool that automatically surfaces what people want built from Reddit and HN' on r/SideProject, r/IndieHackers, r/webdev
- Show HN post with technical write-up of the NLP + vector clustering pipeline
- Product Hunt launch with a GIF demo of the live pain feed
- Twitter/X thread showing the most surprising pain clusters found by the tool
- Direct outreach to 3-5 indie hacker newsletters for feature consideration



08 — TESTING & ACCEPTANCE CRITERIA

How we know it works

Functional acceptance criteria

Area	Criterion
Ingestion	System ingests minimum 500 new posts per day from Reddit + HN combined with zero manual intervention
Classification accuracy	LLM classifier correctly identifies pain signals vs noise with >80% accuracy on a manually labelled test set of 100 posts
Clustering quality	Semantically identical pains (manually verified) are grouped into the same cluster in >85% of test cases
Gap scoring	Gap scores are reproducible — running the same cluster through the scorer twice returns the same score ± 0.1
Dashboard load time	Pain cluster list loads in under 2 seconds on a standard connection
WebSocket feed	Live feed updates within 5 seconds of a new pain signal being classified
Auth flow	GitHub OAuth login completes in under 3 seconds with zero errors across Chrome, Firefox, Safari
Uptime	All three backend services maintain 99%+ uptime over a 7-day test period before launch

Non-functional requirements

- All user data stored in EU/Canada-compliant AWS regions
- No user data sold or shared with third parties — ever
- Self-hostable via Helm chart — users can run their own instance
- Zero-downtime deployments via Kubernetes rolling updates
- All secrets managed via AWS Secrets Manager — no hardcoded credentials anywhere in the codebase
- Full structured logging across all three services — every request traceable via AWS X-Ray

09 — RISKS & MITIGATIONS

What could go wrong



Reddit API access

Risk	Reddit restricts or kills API access (as in 2023).
Mitigation	Build abstraction layer so any source can be swapped. Prioritise HN as backup. Store raw data locally.

LLM API costs

Risk	Classification costs spiral if volume is high.
Mitigation	Use Ollama locally during development (\$0). Add daily token budget caps in production. Cache classification results.

Low retention

Risk	Users check the dashboard once and don't return.
Mitigation	Weekly email digest keeps users engaged passively. Saved alert notifications bring users back when relevant new clusters emerge.

Cold start problem

Risk	Dashboard looks empty on day one with no data.
Mitigation	Pre-seed with 3 months of historical Reddit + HN data before launch. Users see a full, rich dashboard from day one.

Clustering quality

Risk	Clusters are too broad or too narrow to be useful.
Mitigation	Manual review of first 50 clusters before launch. Tune embedding model and similarity threshold iteratively.



10 — TECHNOLOGY OVERVIEW

The stack at a glance

This is not a technical specification — that lives in a separate document. This section gives stakeholders a high-level view of the technology choices and why they were made.

Component	Role
Go — pain-ingestor service	High-concurrency ingestion from Reddit + HN APIs. Goroutines handle rate limiting and parallel fetching efficiently.
Python FastAPI — classifier service	LLM orchestration, embedding generation, and vector clustering. Python's ML ecosystem is unmatched for this layer.
Node.js + GraphQL — API service	Serves the React dashboard. Handles billing webhooks, email triggers, and all read queries.
React + TypeScript	Dashboard frontend. Real-time pain feed via WebSocket. Gap score visualisations via Recharts.
MongoDB Atlas	Raw post storage. Schemaless — each platform (Reddit, HN) has different metadata structure.
PostgreSQL (AWS RDS)	Structured data — users, clusters, billing, gap scores.
Qdrant	Vector store for semantic similarity clustering. Core technical differentiator.
Redis (AWS ElastiCache)	Job queue between ingestor and classifier. Session cache and rate limiting.
Kubernetes (AWS EKS) + Helm	Container orchestration. Self-hostable via Helm chart.
Terraform	All AWS infrastructure as code. Reproducible across dev, staging, prod.
GitHub Actions	CI/CD pipeline per service — test, build, push to ECR, deploy to EKS.
Auth0 + GitHub OAuth	User authentication. JWT access tokens with refresh rotation.
Stripe	Billing — free tier and pro tier (\$12/month).



AWS X-Ray + Prometheus + Grafana

Distributed tracing and metrics. Full observability across all services.